

C868 – Software Capstone Project Summary

Task 2 – Section C



Capstone Proposal Project Name: MeetUp - A Meeting Management Application

Student Name: Ella B. Upchurch

Table of Contents

Application Design and Testing.....	4
Design Document.....	4
Class Design.....	4
UI Design.....	6
Unit Test Plan.....	7
Introduction.....	7
Purpose.....	7
Overview.....	7
Test Plan.....	8
Items.....	8
Features.....	8
Deliverables.....	8
Tasks.....	8
Needs.....	8
Pass/Fail Criteria.....	9
Specifications.....	9
Procedures.....	11
Results.....	12
C4. Source Code.....	16
C5. Link to Live Version.....	16
User Guide.....	17
Introduction.....	17
Logging In.....	21
Navigating the Application.....	22
Main Directory.....	22
Customers Screen.....	23
Searching for Customers.....	23
Adding A Customer Profile.....	24
Updating a Customer.....	26
Deleting a Customer.....	28
Meetings Screen.....	30
Searching for Meetings.....	30
Adding A Meeting.....	31
Updating a Meeting.....	34
Deleting a Meeting.....	36
Reports.....	38

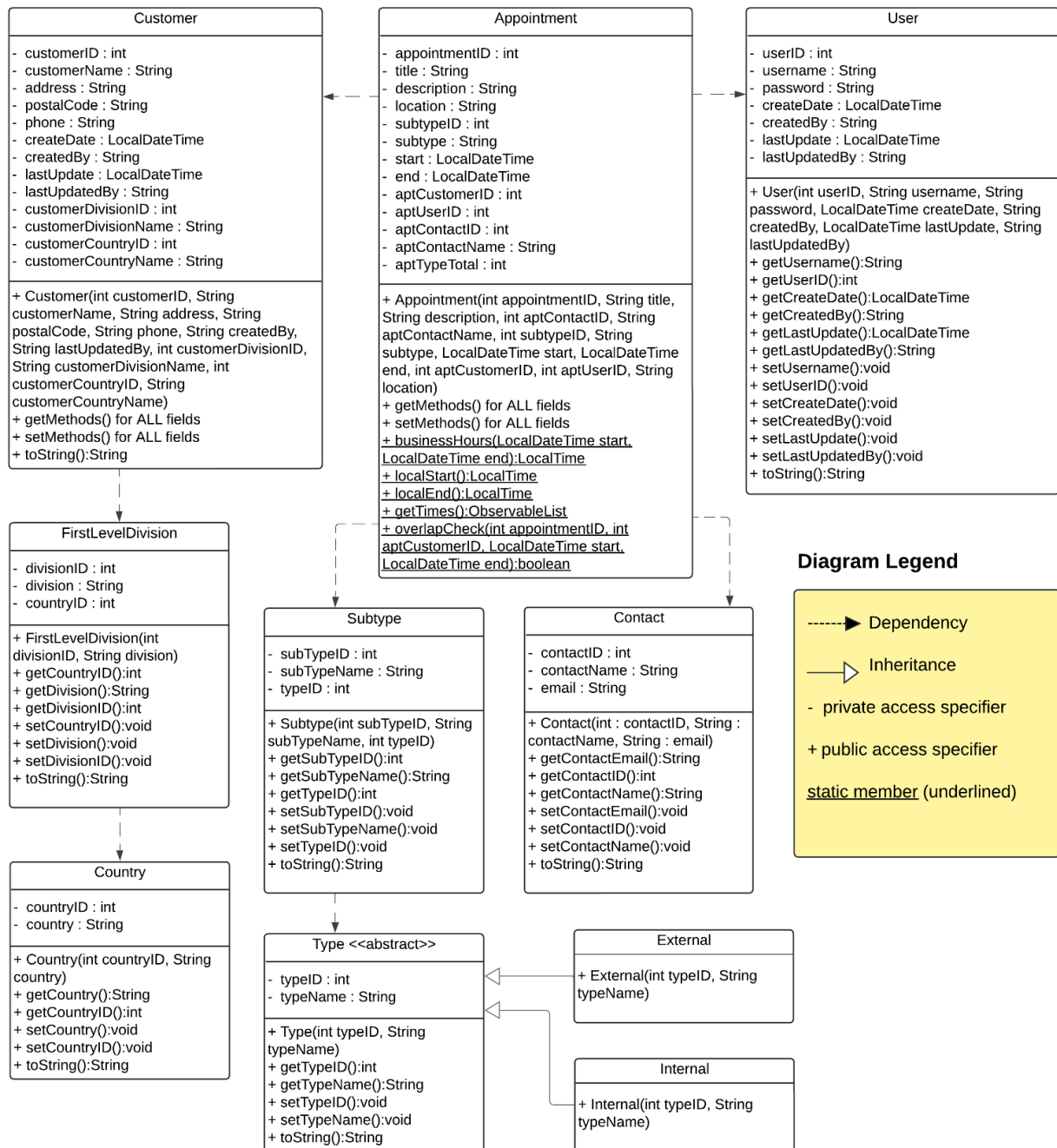
Application Design and Testing

Design Document

Class Design

The Diagram below depicts the class structure of the MeetUp application. Appointment, Customer, and Subtype are the primary classes. The Appointments class holds “Meeting” data but has been left as “Appointments” for efficiency and code reuse. It has many dependencies as it uses data from many other classes, such as userID or subtypeID. The Type class is abstract and is extended by the Internal and External classes. Subtypes are dependent on their associated type, Internal or External. The User and Contact classes exist independently.

The minus sign (-) in the diagram represents private fields, while the plus (+) represents public fields. Dashed arrows mark dependencies, and inheritance is marked with unfilled arrows. Static members are underlined. There is a visual key within the diagram to assist in navigating the classes.



UI Design

Below are the high-fidelity wireframes for the Customers and Meeting screens. The Customers screen features a table of customers that can be filtered using the search bar. It contains buttons for adding, updating, and deleting customer profiles. The screen also includes a 'Return to Main' button that takes the customer to the main directory screen. Like the Customers screen, the Meetings screen contains a search bar, a return button, and add, update, and delete buttons. In addition to these features, the screen has radio buttons allowing users to filter meetings by type: Internal or External.

Customers

ID	Name	Phone	Address	Country	Division
1	Sample Doe	123-456-7890	123 Bird Way	USA	Colorado
2	Example Johnson	555-555-5555	6890 Happy Lane	UK	Scotland
3	Tester Wilson	888-777-6666	868 Capstone Drive	Canada	Northwest

Return to Main

Add

Update

Delete

Meetings

☒ All Appointments ☐ Internal ☐ External

ID	Title	Description	Location	Contact	Type	Start	End	Customer ID	User ID
1	Sample	...	Virtual	3	Q&A	09:00 4/15	09:30 4/15	1	1
2	Example	...	RM 305	2	Project Brief	12:30 4/18	13:30 4/18	2	1

Return to Main

Add

Update

Delete

Unit Test Plan

Introduction

Purpose

The developers created the unit test plan to test the functionality of the Login screen. It also tested the User class methods and UserDAO data accessing methods. In each iteration, a set of login credentials was entered and tested for functionality. The application passed all five iterations of testing.

Overview

This unit test was performed manually in the application GUI. It involves entering a series of correct and incorrect credentials into the username and password fields of the login screen. After each set of credentials is entered, the user attempts to log in. By observing the results of this test, we can determine whether the login code is functioning as intended. Each login attempt is logged into a file labeled *login_activity.txt*. The file can be referenced during testing to determine pass or fail results.

Test Plan

Items

To test whether a user with a valid username or password can log in to the application, we need a model user instance. The user instance should have a valid username and password and be correctly stored in the database. In this unit test, the username will be “test” and the password will be “test.”

Features

Testing will be completed using the Login screen GUI. It will utilize the username and password fields and the login button. Immediate results will be shown through GUI alerts and console output. The `login_activity.txt` can be checked after the actions to confirm the results.

Deliverables

- Console output screenshots
- Data in the `login_activity.txt` file

Tasks

1. Run the application
2. Type username and password credentials into the appropriate fields
3. Press the login button
4. Observe alerts and console output
5. Document the results of the test iteration

Needs

- IntelliJ IDEA Community 2023
- MySQL Database Connection
- Java Packages: `Main.java`, `User.java`, `UserDAO.java`, `JDBC.java`,
`LoginController.java`, `LoginScreen.fxml`
- `login_activity.txt` file

Pass/Fail Criteria

If the application allows the user to log in with the correct credentials, the test is a pass. If it allows a user to log in with incorrect credentials, it is considered a failure. Therefore, if the

expected and actual results in the testing table match, the test will be considered successful. Otherwise, it will be considered a failure.

Specifications

The following code is from the `LoginController.java` package. It contains the `onLoginButtonClick()` method, which gathers the input username and password and cross-checks the database for a matching user. If the credentials cannot be verified, it displays an alert informing the user of the issue. It records login attempts in the `login_activity.txt` file with the date, time, username, and attempt status.

```

/**
 * validates username and password, displays error messages when incorrect credentials are entered
 * logs login attempts in login_activity.txt file
 *
 * @param actionEvent login button pressed
 * @throws IOException
 * @throws SQLException
 */
1 usage  ▲ yellowellaphant
@FXML
public void onLoginButtonClick(ActionEvent actionEvent) throws IOException, SQLException {
    String User_Name = usernameField.getText();
    String Password = passwordField.getText();

    if (User_Name.isBlank()) { //blank username field
        helper.Alerts.displayAlert(alertType: 1);
    } else if (Password.isBlank()) { //blank password field
        helper.Alerts.displayAlert(alertType: 2);
        loginSuccess = false;
        loginActivity();
    } else if (!passwordValidation(Password) && !usernameValidation(User_Name)) { //username + password incorrect
        helper.Alerts.displayAlert(alertType: 3);
        loginSuccess = false;
        loginActivity();
    } else if (!usernameValidation(User_Name)) { //username incorrect
        helper.Alerts.displayAlert(alertType: 3);
        loginSuccess = false;
        loginActivity();
    } else if (!passwordValidation(Password)) { //password incorrect
        helper.Alerts.displayAlert(alertType: 3);
        loginSuccess = false;
        loginActivity();
    } else if (userLogin(User_Name, Password)) { //login successful

        int userID = getUserID(User_Name);
        ObservableList<Appointment> userAppointments = AppointmentDAO.getUserAppointments(userID);

        Parent parent = FXMLLoader.load(Objects.requireNonNull(getClass().getResource("name: "/wgu/c195/DirectoryScreen.fxml"))));
        Scene scene = new Scene(parent);
        Stage stage = (Stage) ((Node) actionEvent.getSource()).getScene().getWindow();
        stage.setScene(scene);
        stage.centerOnScreen();
        stage.show();

        loginSuccess = true;
        loginActivity();

        boolean isValid = false;
        for (Appointment appointment : userAppointments) {
            LocalDateTime startTime = appointment.getStart();
            if ((startTime.isAfter(currentTime) || startTime.isEqual(next15Minutes)) &&
                (startTime.isBefore(next15Minutes) || startTime.isEqual(currentTime))) {
                Alert upcomingAppointment = new Alert(Alert.AlertType.WARNING);
                upcomingAppointment.setTitle("Alert");
                upcomingAppointment.setContentText("Appointment " + appointment.getAppointmentID() + " begins at " + appointment.getStart());
                upcomingAppointment.getButtonTypes().clear();
                upcomingAppointment.getButtonTypes().addAll(ButtonType.CANCEL, ButtonType.OK);
                upcomingAppointment.showAndWait();
                isValid = true;
                break; // No need to continue the loop once a valid appointment is found
            }
        }
        if (!isValid) {
            helper.Alerts.displayAlert(alertType: 5);
        }
    }
}
}

```

Procedures

The following test iterations will be used in this unit test:

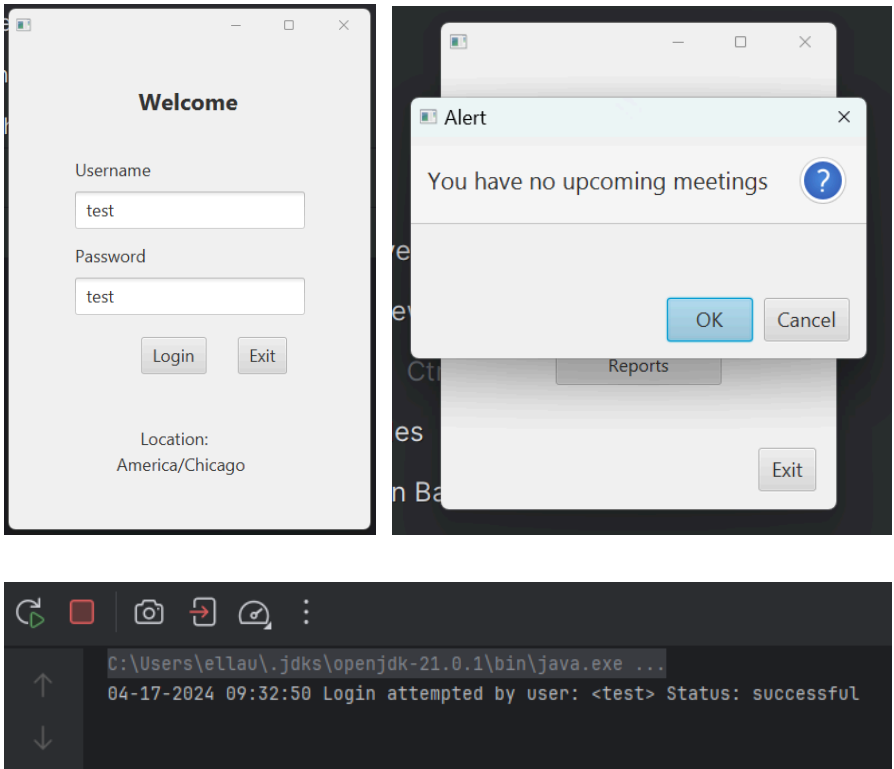
Test Iteration	Task Description	Data Input	Expected Results	Actual Results	Pass/Fail
1	Attempt to log in with the correct username & password	<u>Username:</u> "test" <u>Password:</u> "test"	Login Successful; console reports successful login	Login Successful; console reports successful login	Pass
2	Attempt to log in with empty username & password fields	<u>Username:</u> left empty (null) <u>Password:</u> left empty (null)	The login failed with an error message; no login report in the console.	The login failed with an error message; no login report in the console.	Pass
3	Attempt to log in with incorrect username & password	<u>Username:</u> "incorrect" <u>Password:</u> "incorrect"	The login failed with an error message; the console reports a failed login.	The login failed with an error message; the console reports a failed login.	Pass
4	Attempt to log in with incorrect username & correct password	<u>Username:</u> "incorrect" <u>Password:</u> "test"	The login failed with an error message; the console reports a	The login failed with an error message; the console reports a	Pass

			failed login.	failed login.	
5	Attempt to log in with the correct username & incorrect password	<u>Username:</u> "test" <u>Password:</u> "incorrect"	The login failed with an error message; the console reports a failed login.	The login failed with an error message; the console reports a failed login.	Pass

Results

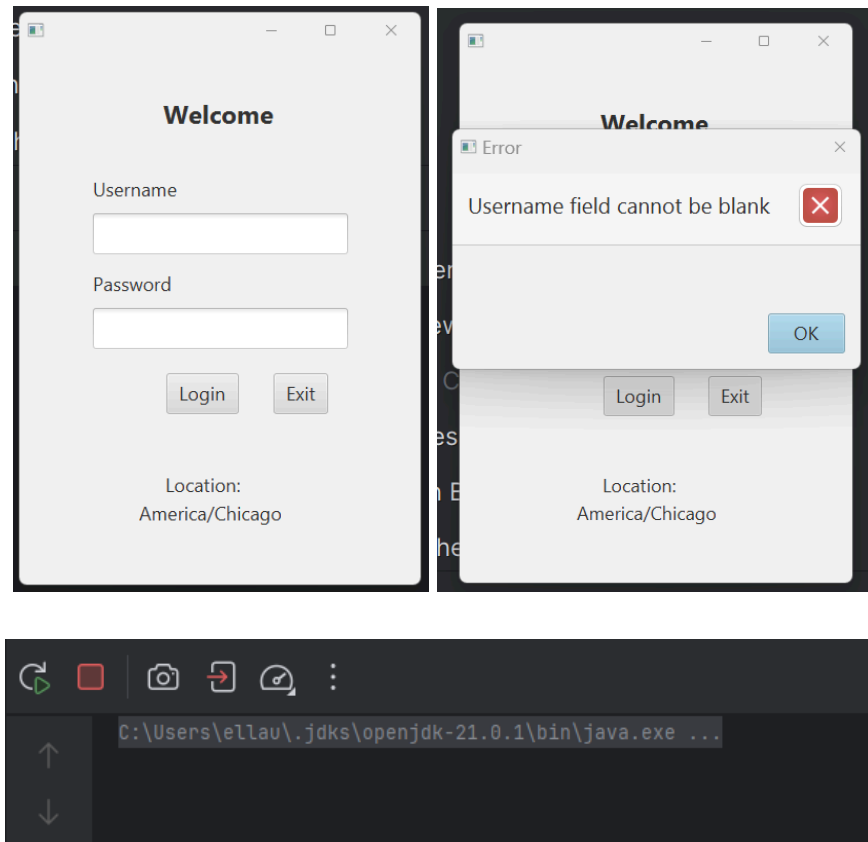
Below are screenshots of the results of each test iteration listed in the Procedure section.

Test Iteration 1:



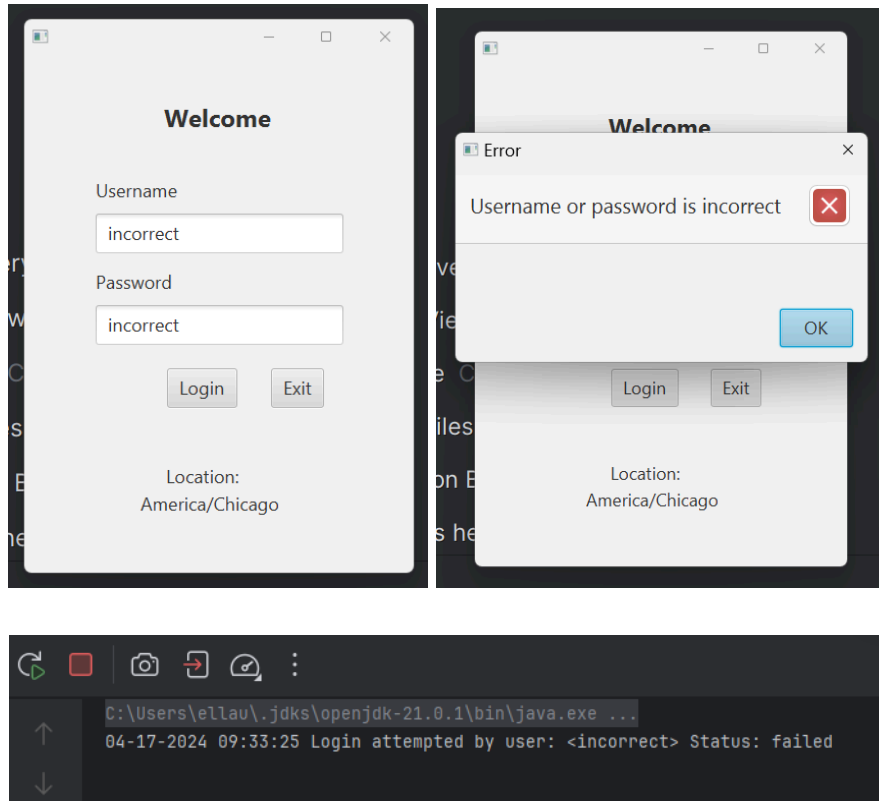
Results: Login Successful; console reports successful login

Test Iteration 2:



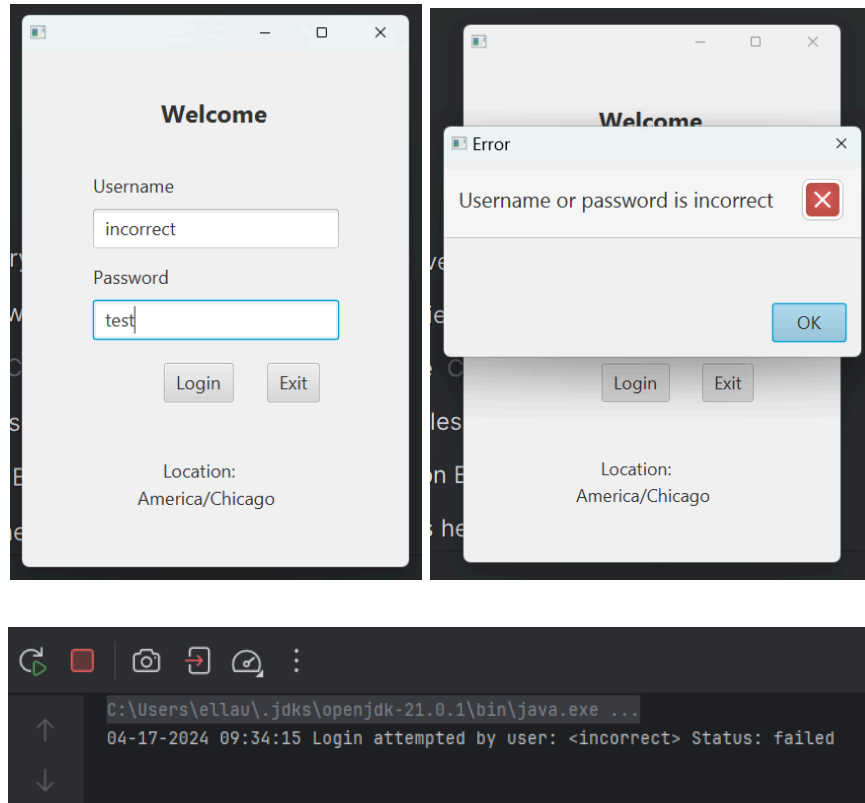
Results: Login failed with an error message; no login report in the console.

Test Iteration 3:



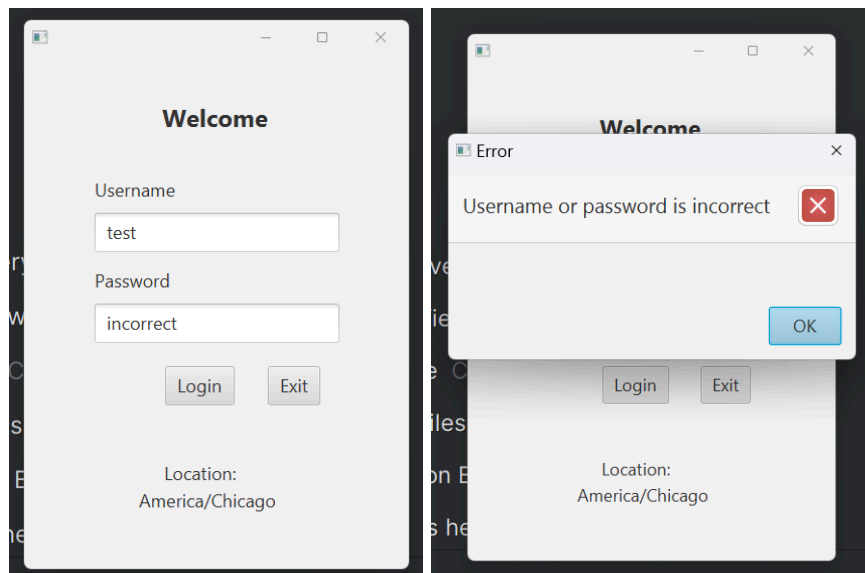
Results: Login failed with an error message; console reports failed login.

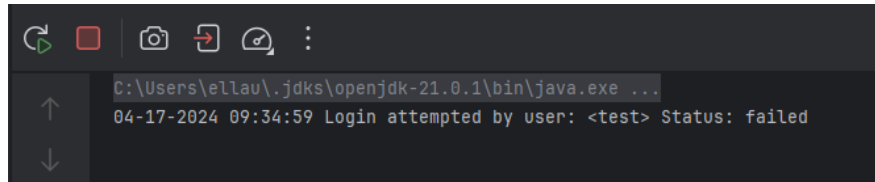
Test Iteration 4:



Results: Login failed with an error message; console reports failed login.

Test Iteration 5:

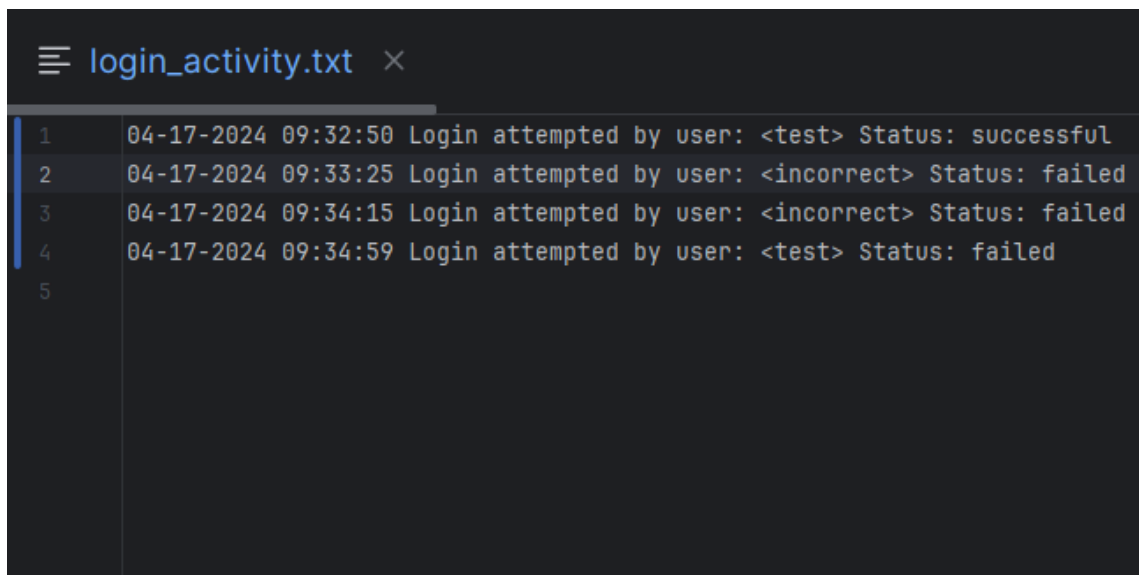




```
C:\Users\ellau\.jdk\openjdk-21.0.1\bin\java.exe ...  
04-17-2024 09:34:59 Login attempted by user: <test> Status: failed
```

Results: Login failed with an error message; console reports failed login.

login_activity.txt results:



```
login_activity.txt  
1 04-17-2024 09:32:50 Login attempted by user: <test> Status: successful  
2 04-17-2024 09:33:25 Login attempted by user: <incorrect> Status: failed  
3 04-17-2024 09:34:15 Login attempted by user: <incorrect> Status: failed  
4 04-17-2024 09:34:59 Login attempted by user: <test> Status: failed  
5
```

Results: login attempt results match data in the *login_activity.txt* file

Source Code

The source code can be found in the attached “C868-WGU-Start” file.

Link to Live Version

There is no live version of this application available.

User Guide

Introduction

This guide will include instructions on setting up the application on your local machine, logging in, and utilizing all its features. It will also include links to all necessary software and packages to run the application, including meeting creation, customer profile creation, and report use.

Download and Installation:

(Skip this section if you are utilizing the lab environment to evaluate this application)

1. Download and Install IntelliJ [IDEA Community](#) version 2023.2.2+
 - a. Scroll down the page to find the Community download
2. Download and Install [MySQL Server and MySQL Workbench](#) version 8.0+
 - a. Download the larger version; usually the second option
 - b. When using the setup wizard, select MySQL Server and MySQL Workbench as products to install
 - c. When prompted, choose a password and remember it. You will need it in the next section.
3. Download [JDK](#) version 17+
 - a. Once downloaded, unzip the file and save it to an accessible location
4. Download [JavaFX](#) version 17+
 - a. Once downloaded, unzip the file and save it to an accessible location
5. Download [MySQL Connector-J](#) version 8.3+
 - a. Select “Platform Independent” as the Operating System
 - b. Download the larger version; usually the second option

- c. Once downloaded, unzip the file and save it to an accessible location

Setting Up MySQL Workbench:

(Skip this section if you are utilizing the lab environment to evaluate this application)

1. Click on *Local Instance* connection in my connections
2. Enter the password you created during the installation
3. Select the *Administration* tab (next to *Schemas*) and click *Users and Privileges*
4. Set the following fields in the Login tab:
 - a. Login Name: sqlUser
 - b. Authentication Type: SHA256 Password
 - c. Limit to Hosts Matching: localhost
 - d. Password: Passw0rd!
5. Open the Administrative Roles tab
 - a. Select the DBA checkbox
6. Click Apply and close the *Local Instance*
7. Once on the homepage, click the + button next to *MySQL Connections* to create a new connection.
8. Fill/Change in the following fields:
 - a. Connection Name: C868JDBC (you may name it something else if you'd like)
 - b. Hostname: localhost
 - c. Username: sqlUser
9. Click *Test Connection*
10. Enter the password and click *ok*

11. The pop-up should say, “Successfully made the MySQL connection.” If it doesn’t, start from the first step.

Populate the Database:

1. Locate the Database DDL & DML files. They are attached separately.
2. Open the C868JDBC connection you created in the last section; you may have given it an alternate name (If you are evaluating using the lab environment, use the existing connection and skip to step 7)
3. Select the *Schemas* tab. You should see one database called *sys*. Take note of the symbol to the left of it (this is the database symbol).
4. Click the database symbol with a + on the bottom left corner (this creates a new database) and fill in the following fields:
 - a. Name: *client_schedule*
5. Click *Apply* and then *Finish*
6. There should now be a database called *client_schedule* above the *sys* database in the *schemas* tab
7. Create Database Tables:
 - a. Copy the whole script from the C868-DB-DDL document.
 - b. Select *File > New Query Tab*
 - c. Paste the C868-DB-DDL script into the populated query tab
 - d. Click the lightning bolt button to execute the query
 - e. To check if the query was successful, expand the Tables section of *client_schedule*
 - f. The following tables should have been added:
 - i. appointments

- ii. contacts
- iii. countries
- iv. customers
- v. first_level_divisions
- vi. subtypes
- vii. types
- viii. users

8. Populate Table Data:

- a. Copy the whole script from the C868-DB-DML document.
- b. Select *File > New Query Tab*
- c. Paste the C868-DB-DML script into the populated query tab
- d. Click the lightning bolt button to execute the query
- e. To check if the query was successful, populate a new query tab and insert this line: “`SELECT * FROM client_schedule.appointments;`”
- f. If you see appointment data, the table data has been populated successfully

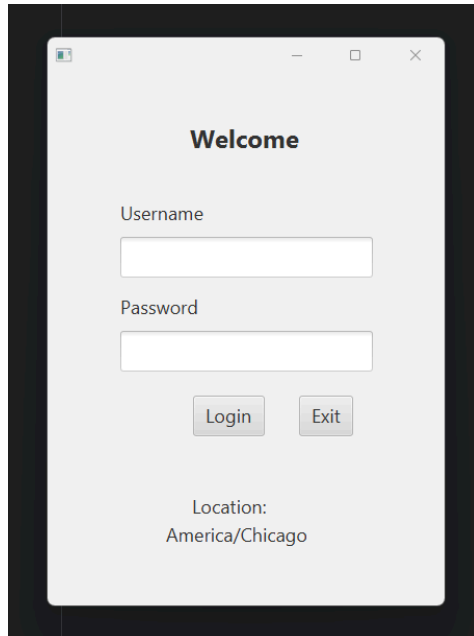
Setting Up the Application in IntelliJ:

- 1. Unzip the C868-WGU-Start file and open it in IntelliJ
- 2. Navigate to *File > Project Structure*
 - g. select *Libraries* under *Project Settings*
 - h. click the + and choose *Java*
 - i. Locate and open the mysql-connector-j file you downloaded
 - j. Select the jar file. It will be formatted like this: mysql-connector-j-8.3.0.jar
 - k. Add it to all modules (shift-click) and click OK

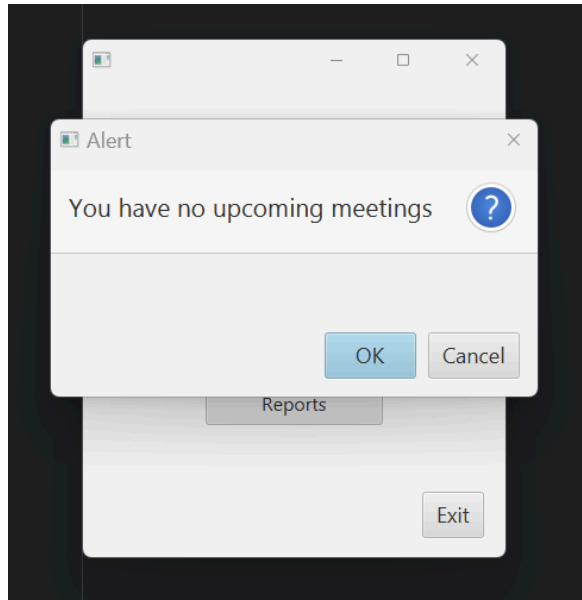
3. Locate and open the *Main* class in the *wgu.c195* folder
4. Run the application

Logging In

1. *To use the application, you must have a valid username and password. You can input these on the Login screen shown below.*



2. *Enter the following credentials:*
 - a. *Username: admin*
 - b. *Password: admin*
3. *Press the “Login” button to continue*
4. *Upon login, an alert will appear that informs you about any appointments you have in the next 15 minutes. If you have no appointments scheduled in the next 15 minutes, you will receive the alert shown below.*

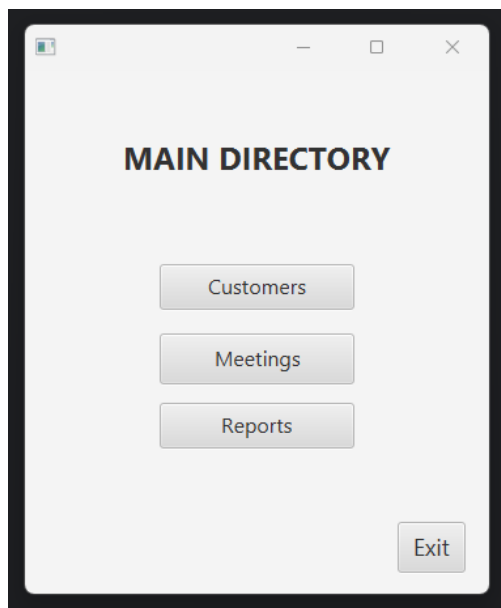


Dismiss this alert using the OK or Cancel button.

Navigating the Application

Main Directory

- 1. Once logged in, you will be taken to the Main Directory screen*



- 2. You can use the buttons to navigate various screens or exit the application.*

Customers Screen

Searching for Customers

3. After clicking the “Customers” button, you will be taken to the Customers screen.

Customers

Search by Name or ID

ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey
2	Crabb Accounting	11-445-910-2135	245 Algebra Way	AF19B	UK	Scotland
3	SuperPups INC	874-916-2671	48 Retriever Ma...	28198	Canada	Northwest Territor...

Return to Main

Add

Update

Delete

4. Typing a full or partial name or ID will filter customer results in the table:

Customers

med

ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey

<

>

Return to Main

Add

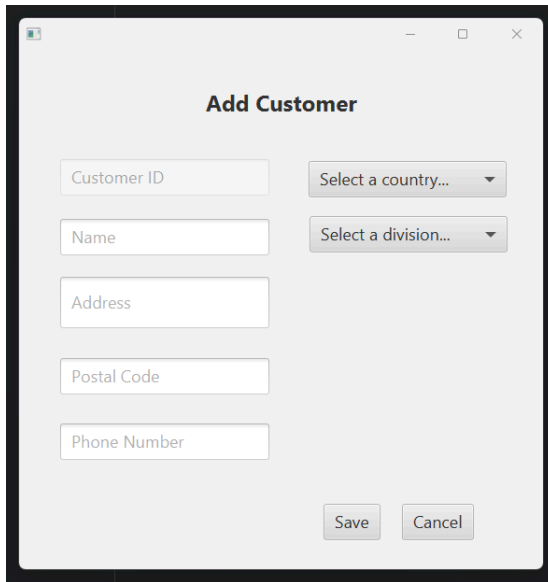
Update

Delete

5. The “Return to Main” button directs you to the Main Directory screen.

Adding A Customer Profile

6. The “Add” button will take you to the Add Customer screen.

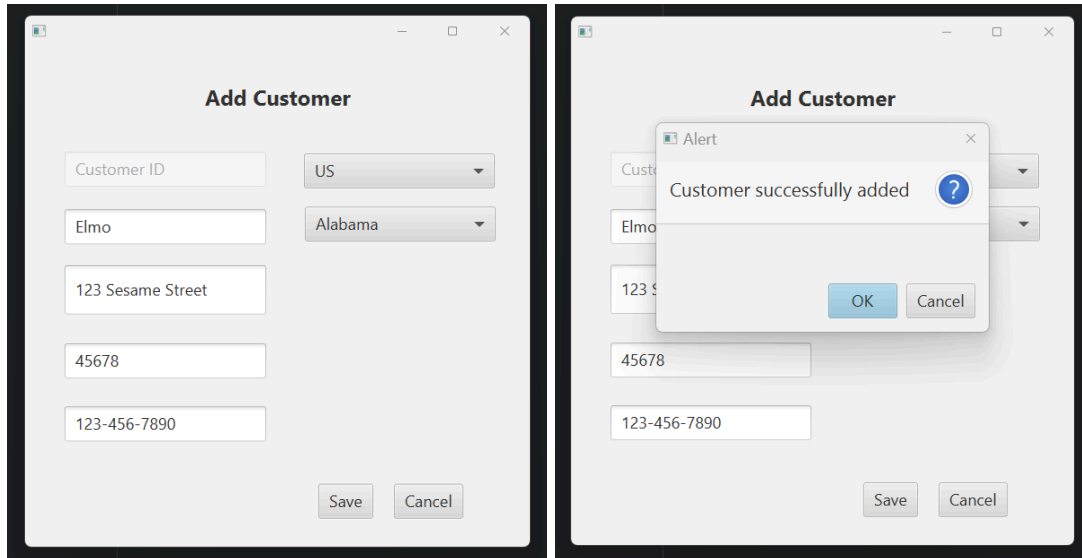


The screenshot shows a web browser window with a form titled "Add Customer". The form contains several input fields and two dropdown menus. The fields are labeled "Customer ID", "Name", "Address", "Postal Code", and "Phone Number". The "Customer ID" field is disabled. The "Select a country..." dropdown menu is open, showing a list of countries. The "Select a division..." dropdown menu is also open, showing a list of divisions. At the bottom of the form, there are "Save" and "Cancel" buttons.

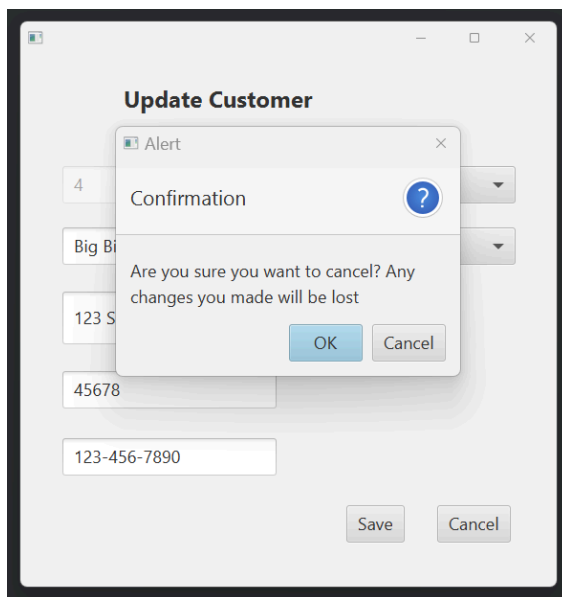
7. Here, you may input Customer information such as name, phone number, or country.

All fields must be completed to save the customer profile. *Note: The customer ID field is disabled, and the country must be selected before the division*

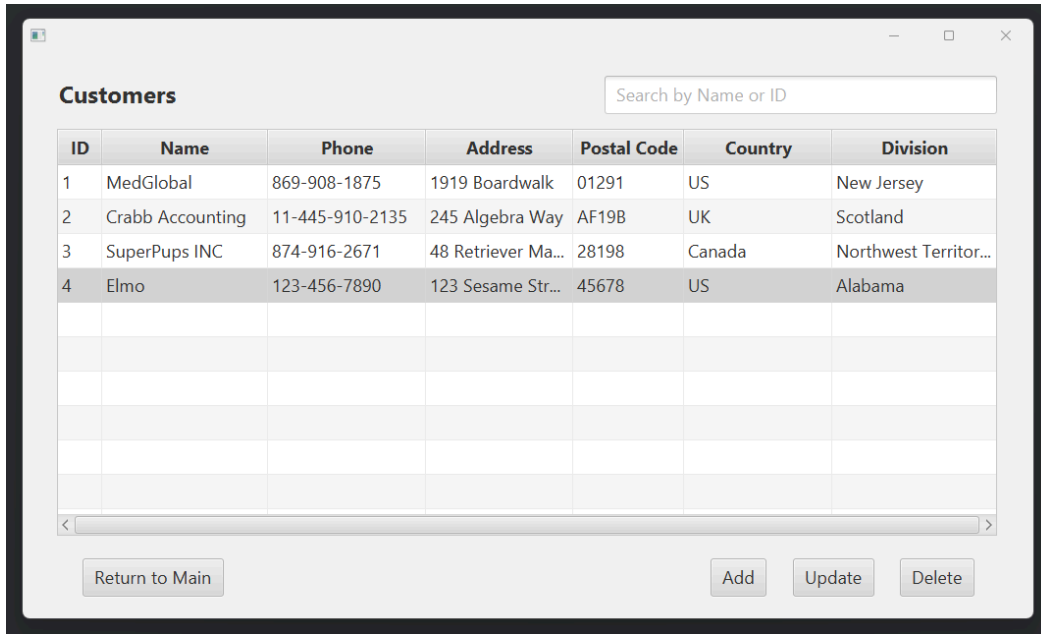
8. Once all fields are completed, clicking the “Save” button will bring up a confirmation prompt, save the new customer profile in the database, and send you back to the Customers screen.



9. Pressing the “Cancel” button will bring up a confirmation prompt and warn you of lost changes. Pressing “OK” will direct you back to the Customers screen (The same goes for the Update Customer screen)



10. If saved successfully, the customer will now appear in the Customer table.

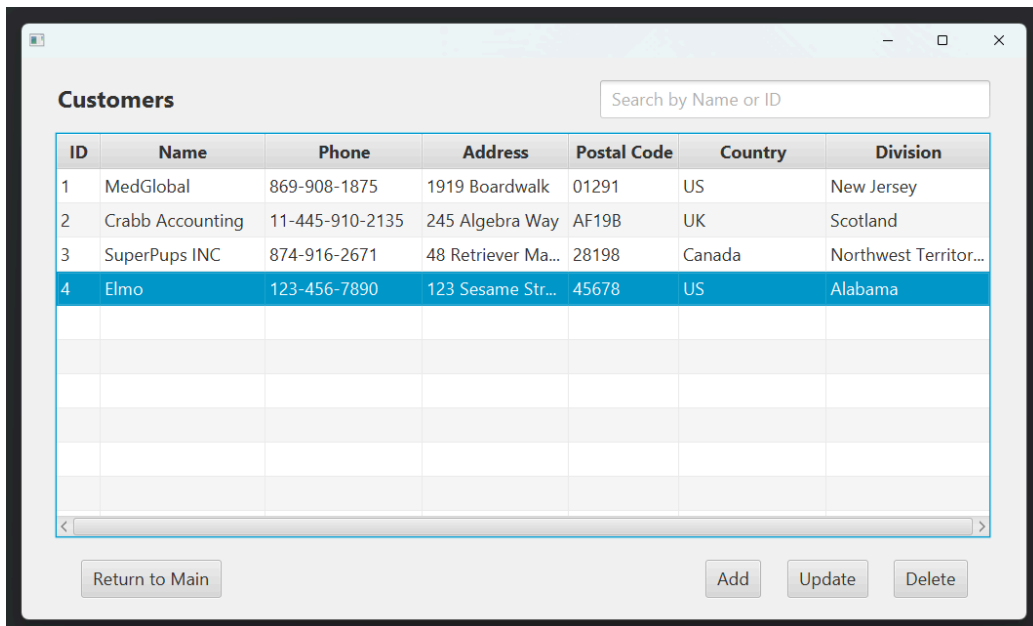


The screenshot shows a web application window titled "Customers". At the top right is a search bar labeled "Search by Name or ID". Below it is a table with 7 columns: ID, Name, Phone, Address, Postal Code, Country, and Division. The table contains 4 rows of data. Below the table is a horizontal scrollbar. At the bottom of the window are three buttons: "Return to Main", "Add", "Update", and "Delete".

ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey
2	Crabb Accounting	11-445-910-2135	245 Algebra Way	AF19B	UK	Scotland
3	SuperPups INC	874-916-2671	48 Retriever Ma...	28198	Canada	Northwest Territor...
4	Elmo	123-456-7890	123 Sesame Str...	45678	US	Alabama

Updating a Customer

11. Select the customer you would like to update and press the "Update" button.



This screenshot is identical to the previous one, but the row for the customer "Elmo" (ID 4) is highlighted in blue, indicating it is selected. The "Update" button at the bottom right is now the primary action.

ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey
2	Crabb Accounting	11-445-910-2135	245 Algebra Way	AF19B	UK	Scotland
3	SuperPups INC	874-916-2671	48 Retriever Ma...	28198	Canada	Northwest Territor...
4	Elmo	123-456-7890	123 Sesame Str...	45678	US	Alabama

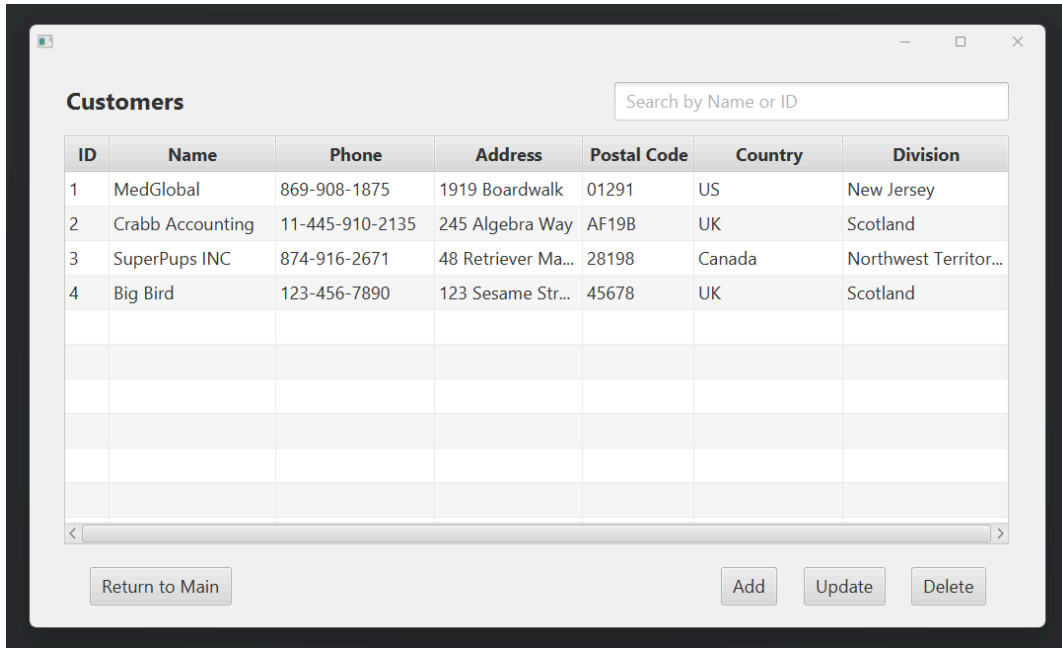
12. You will be directed to the Update Customer screen. Here, you may make changes to the customer profile.

The image shows two side-by-side screenshots of a web application window titled "Update Customer". Each window contains a form with the following fields: a text input for ID (value: 4), a dropdown menu for Country (values: US, UK), a text input for Name (values: Elmo, Big Bird), a dropdown menu for State (values: Alabama, Scotland), a text input for Address (value: 123 Sesame Street), a text input for Zip (value: 45678), and a text input for Phone (value: 123-456-7890). At the bottom right of each form are "Save" and "Cancel" buttons.

13. Pressing the “Save” button will bring up a confirmation prompt and update the database.

The image shows a screenshot of the "Update Customer" form with an "Alert" dialog box overlaid. The dialog box has a title bar "Alert" and a close button. The message inside says "Customer successfully updated" with a blue question mark icon. At the bottom of the dialog are "OK" and "Cancel" buttons. The form in the background is partially obscured but shows the same fields as the previous screenshots, with the "Save" and "Cancel" buttons visible at the bottom.

14. The updated information will now appear in the Customer table.

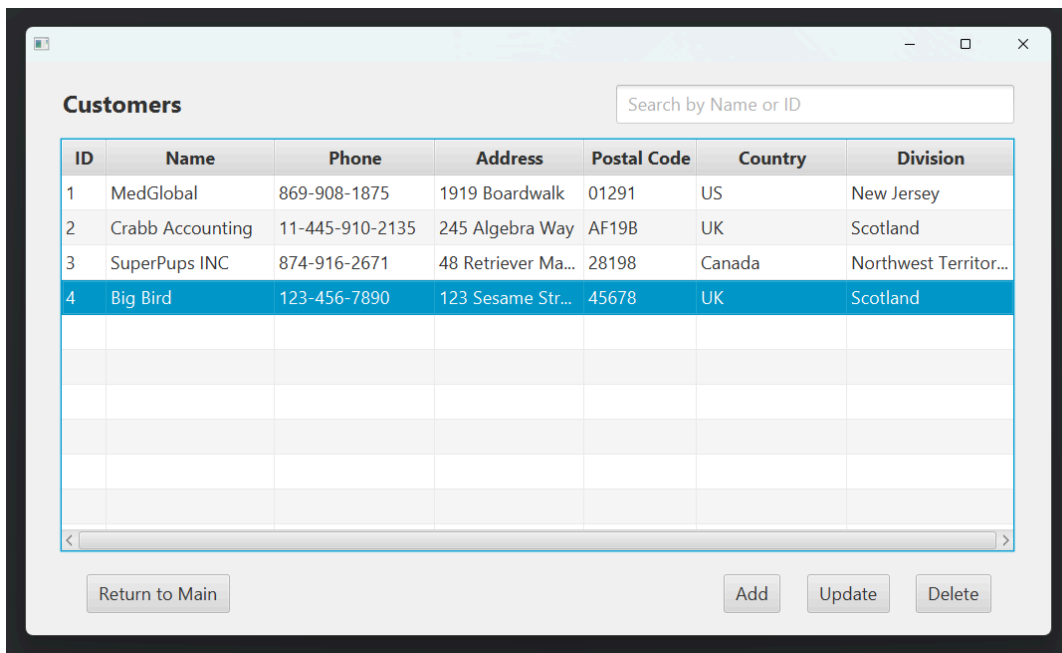


The screenshot shows a web application window titled "Customers". At the top right is a search bar labeled "Search by Name or ID". Below it is a table with 7 columns: ID, Name, Phone, Address, Postal Code, Country, and Division. The table contains 4 rows of data. Below the table is a horizontal scrollbar. At the bottom of the window are three buttons: "Return to Main", "Add", "Update", and "Delete".

ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey
2	Crabb Accounting	11-445-910-2135	245 Algebra Way	AF19B	UK	Scotland
3	SuperPups INC	874-916-2671	48 Retriever Ma...	28198	Canada	Northwest Territor...
4	Big Bird	123-456-7890	123 Sesame Str...	45678	UK	Scotland

Deleting a Customer

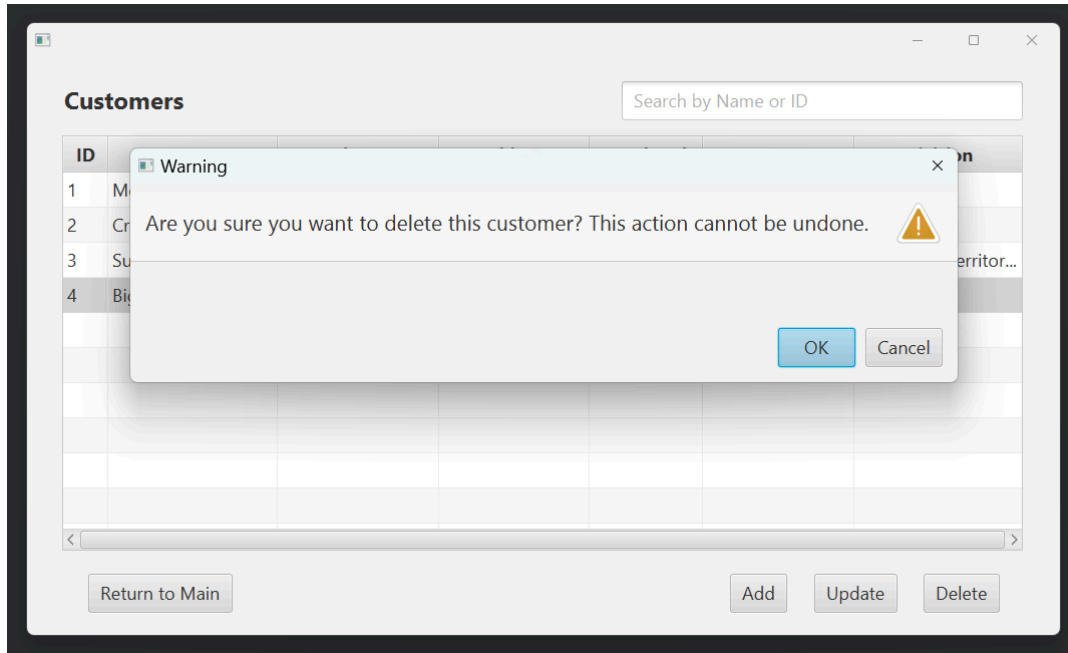
1. *Select the customer you would like to delete*



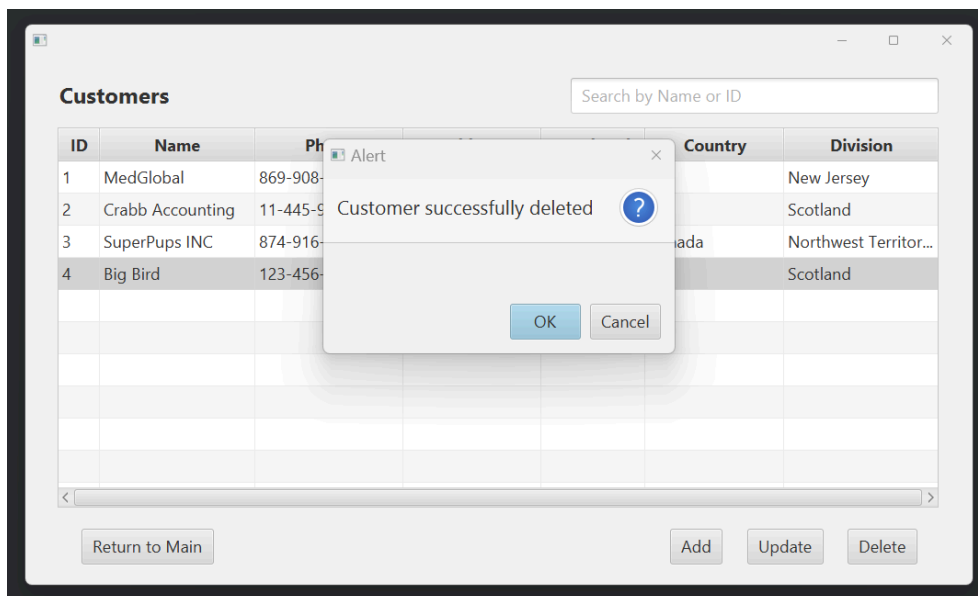
This screenshot is identical to the previous one, but the fourth row of the table (ID 4, Big Bird) is highlighted in blue, indicating it is selected.

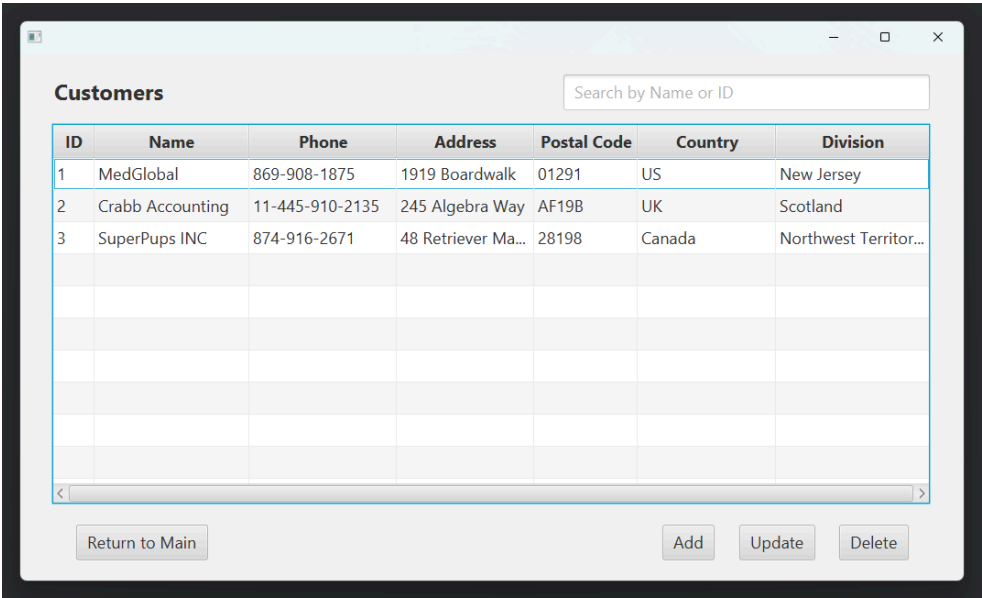
ID	Name	Phone	Address	Postal Code	Country	Division
1	MedGlobal	869-908-1875	1919 Boardwalk	01291	US	New Jersey
2	Crabb Accounting	11-445-910-2135	245 Algebra Way	AF19B	UK	Scotland
3	SuperPups INC	874-916-2671	48 Retriever Ma...	28198	Canada	Northwest Territor...
4	Big Bird	123-456-7890	123 Sesame Str...	45678	UK	Scotland

2. *A confirmation prompt will appear. Select "OK" to permanently delete the customer.*



3. *The customer will no longer appear on the Customer table.*

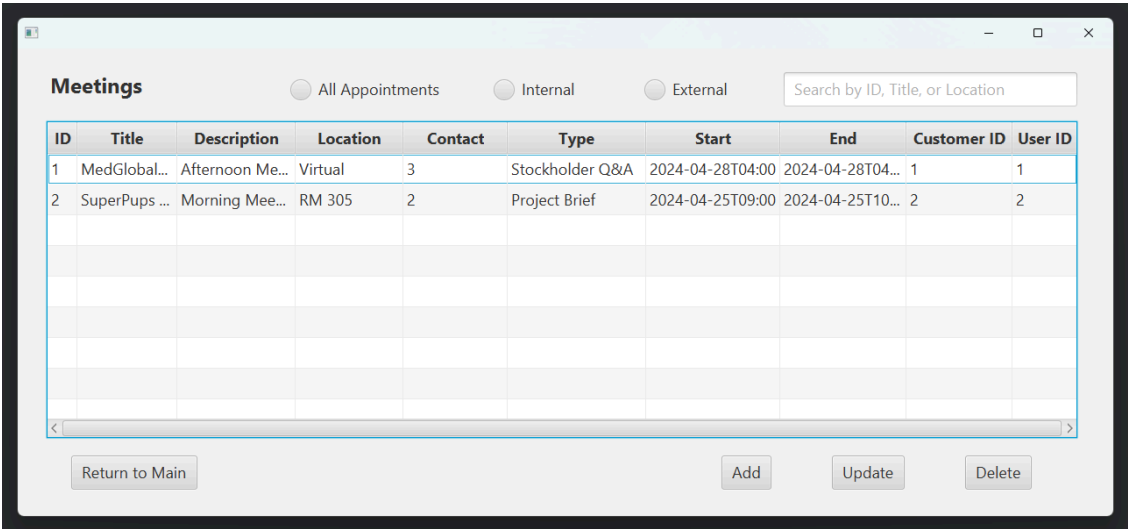




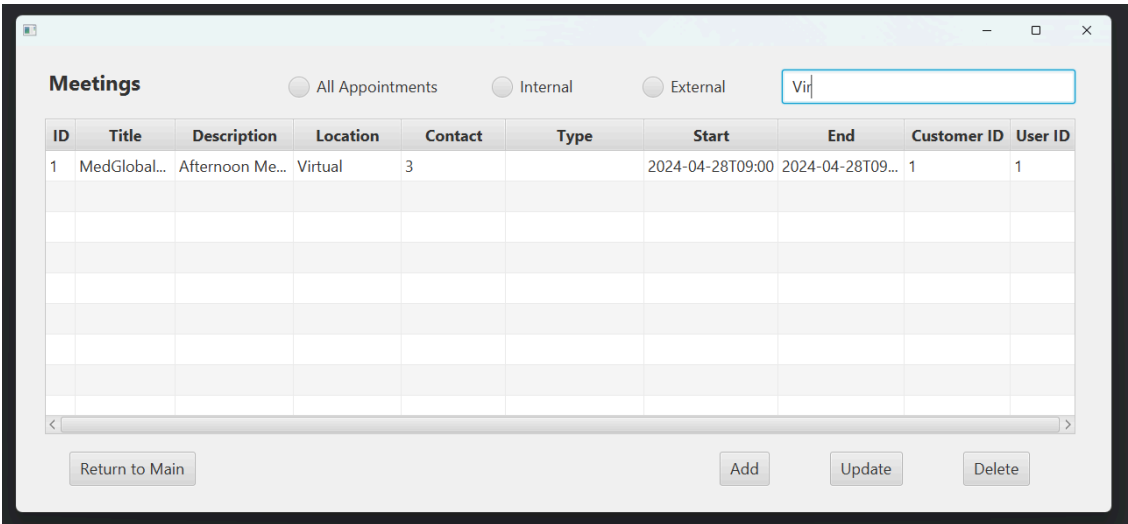
Meetings Screen

Searching for Meetings

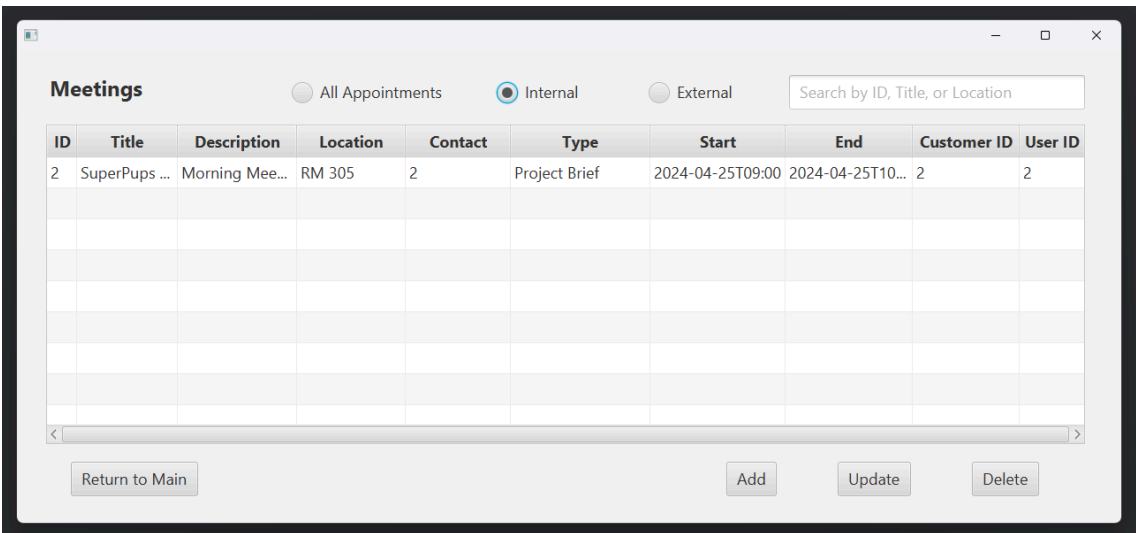
- 1. After clicking the “Meetings” button, you will be taken to the Meetings screen.



- 2. Typing a full or partial ID, title, or location will filter meeting results in the table:



3. The radio buttons can be used to filter meetings by type: Internal or External.



4. Clicking the “Return to Main” button will direct you back to the Main Directory screen.

Adding A Meeting

1. Clicking the “Add” button will take you to the Add Meeting screen.

The screenshot shows a web application window titled "Add Meeting". The form is organized into two main columns. The left column contains text input fields for "Meeting ID" (which is disabled), "Title", "Description", "Location", and dropdown menus for "Type" and "Subtype". The right column contains three dropdown menus labeled "Select a Contact", "Select a User", and "Select a Customer". Below these are two date/time pickers for "Start Date/Time" and "End Date/Time", each with a "Select start date" and "Select end date" button. At the bottom right of the form are "Save" and "Cancel" buttons.

2. Here, you may input Meeting information such as location, type, customer, or start time.

***All fields must be completed** in order to save the Meeting profile. Note: The meeting ID field is disabled, and you cannot set the start/end times before or after business hours of 8:00 AM - 10:00 PM EST*

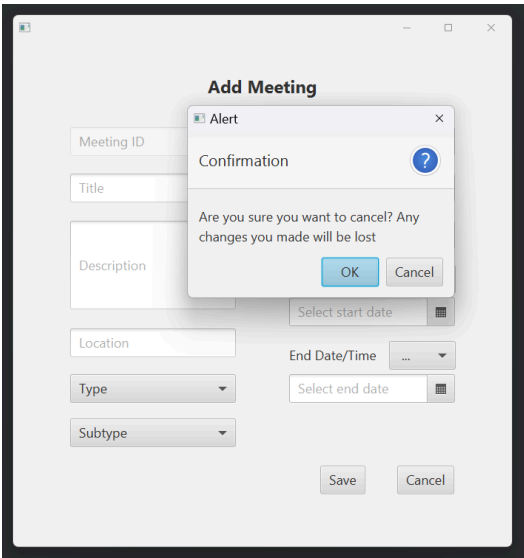
3. Once all fields are completed, clicking the "Save" button will bring up a confirmation prompt, save the new Meeting profile in the database, and send you back to the Meetings screen.

The 'Add Meeting' form is a light gray window with a title bar. It contains several input fields and dropdown menus. On the left, there is a 'Meeting ID' field with the value 'Example', a 'Description' field with 'Sample Text', a 'Location' field with 'Virtual', and a 'Type' dropdown menu with 'Internal' selected. On the right, there is a 'User' dropdown menu with 'Daniel Garcia' selected, a 'Company' dropdown menu with 'Crabb Accounting' selected, a 'Start Date/Time' field with '4/18/2024', and an 'End Date/Time' field with '4/18/2024'. At the bottom right, there are 'Save' and 'Cancel' buttons.

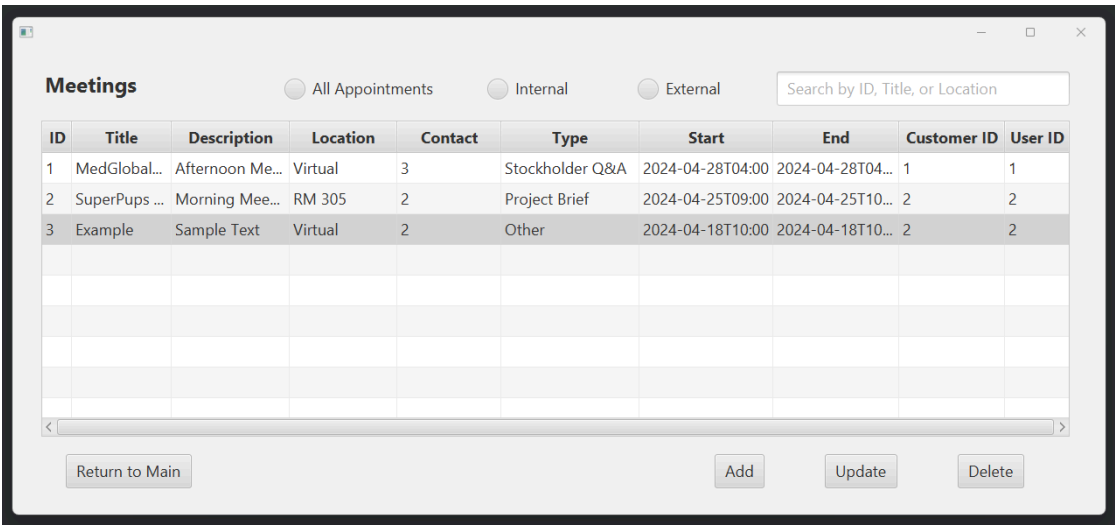
The 'Meetings' screen is a light gray window with a title bar. It features a search bar at the top right with the placeholder text 'Search by ID, Title, or Location'. Below the search bar are three radio buttons: 'All Appointments' (selected), 'Internal', and 'External'. The main content is a table with the following columns: ID, Title, Description, Type, Start, End, Customer ID, and User ID. The table contains three rows of data. At the bottom, there are buttons for 'Return to Main', 'Add', 'Update', and 'Delete'. A modal dialog box is open in the center of the screen, displaying the message 'Meeting successfully added' and 'Alert' in the title bar, with 'OK' and 'Cancel' buttons.

ID	Title	Description	Type	Start	End	Customer ID	User ID
1	MedGlobal...	Afternoon	Stakeholder Q&A	2024-04-28T04:00	2024-04-28T04:00	1	1
2	SuperPups ...	Morning	Project Brief	2024-04-25T09:00	2024-04-25T10:00	2	2
3	Example	Sample	Meeting	2024-04-18T10:00	2024-04-18T10:00	2	2

4. Pressing the “Cancel” button will bring up a confirmation prompt and warn you of lost changes. Pressing “OK” will direct you back to the Meetings screen (The same goes for the Update Meeting screen)

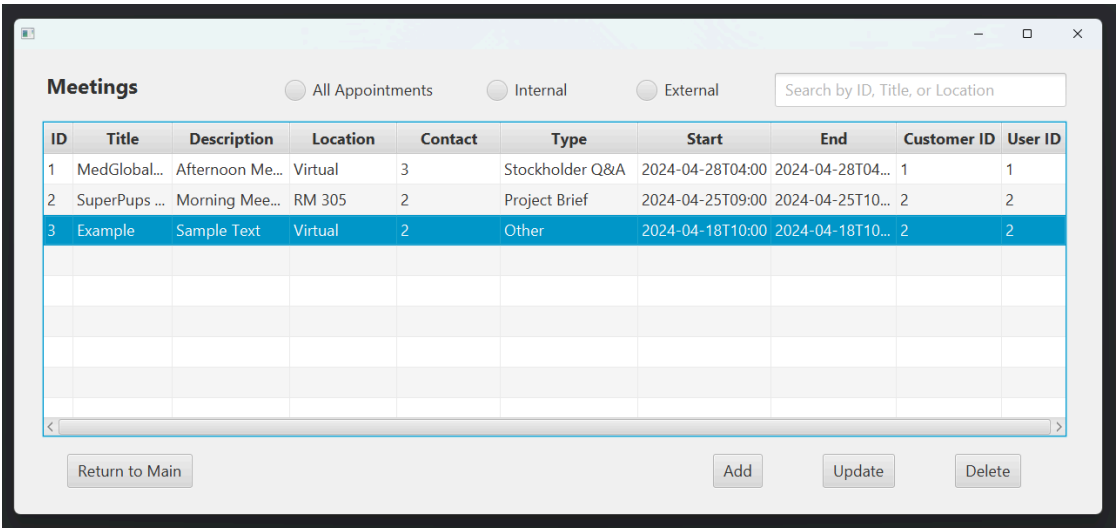


5. *If saved successfully, the Meeting will now appear in the Meeting table.*

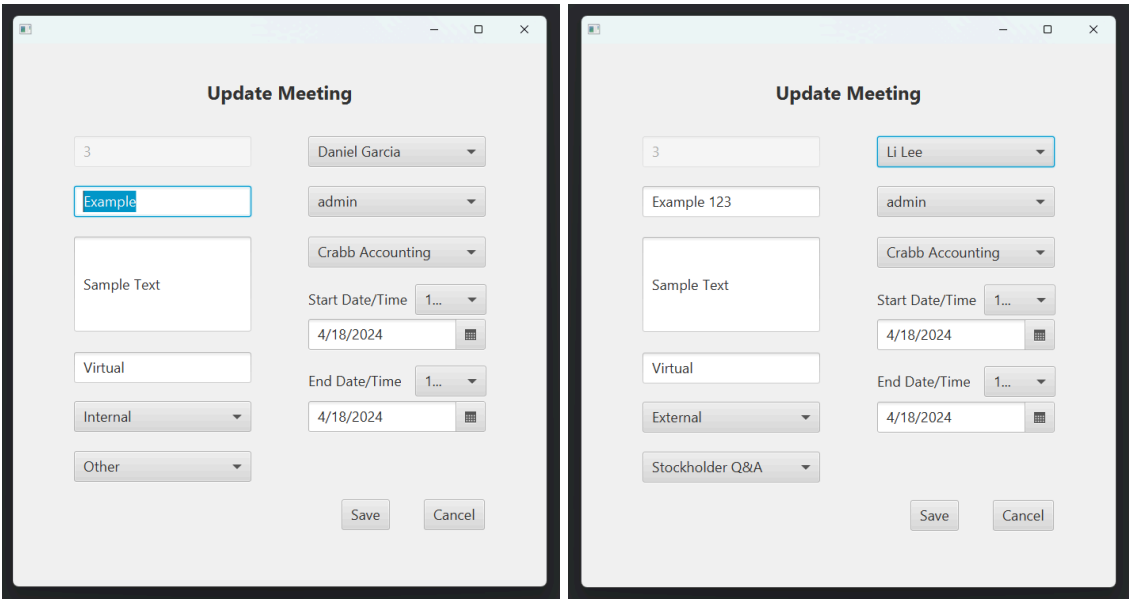


Updating a Meeting

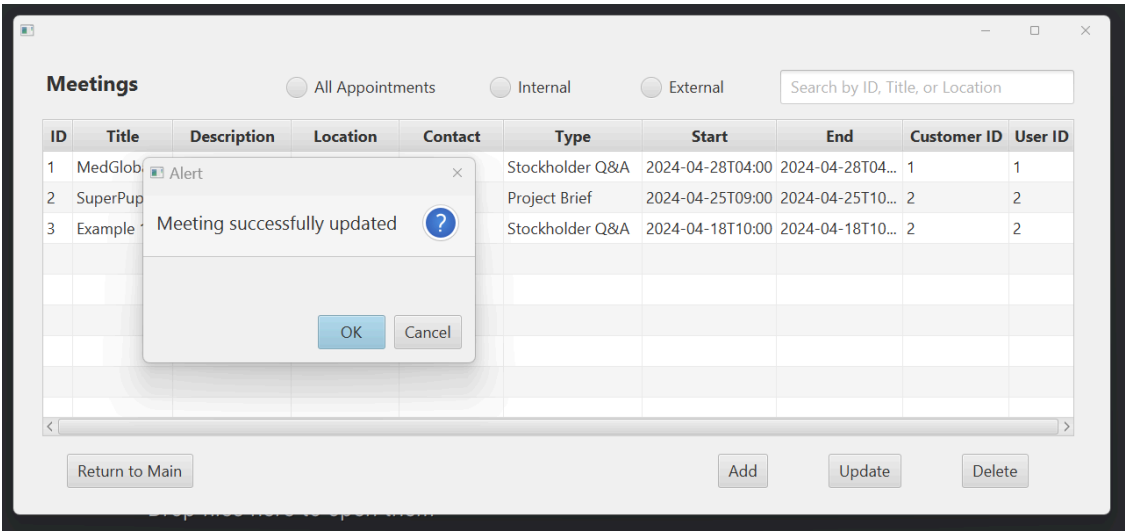
1. *Click on the Meeting you would like to update and press the “Update” button.*



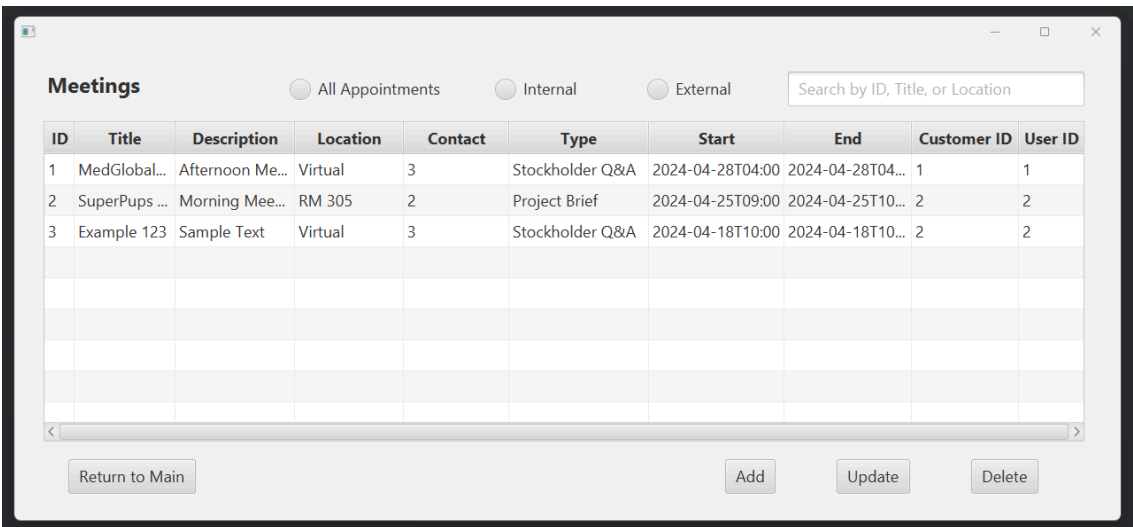
2. You will be directed to the Update Meeting screen. Here, you may make changes to the Meeting profile.



3. Pressing the "Save" button will bring up a confirmation prompt and update the database.

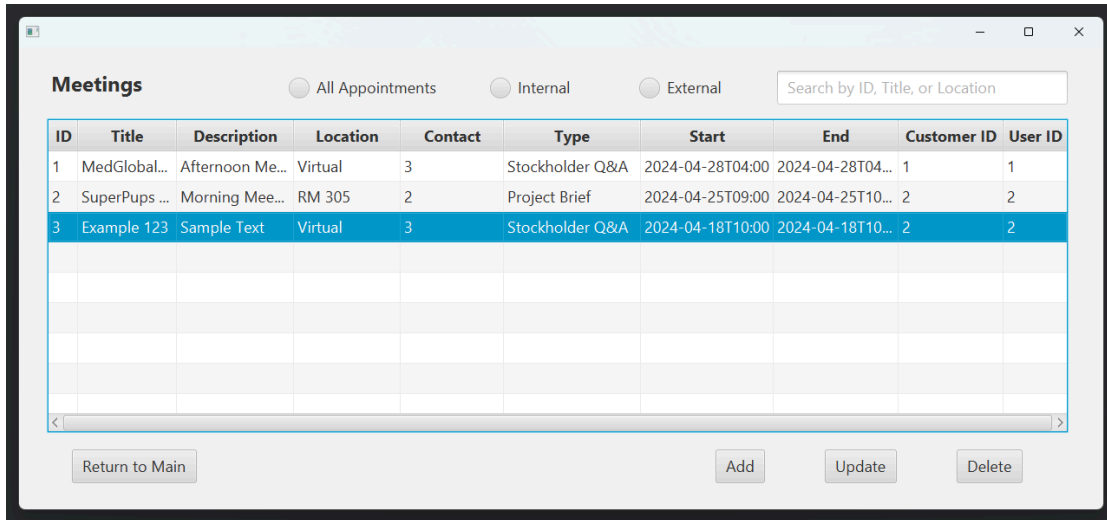


4. The updated information will now appear in the Meeting table.

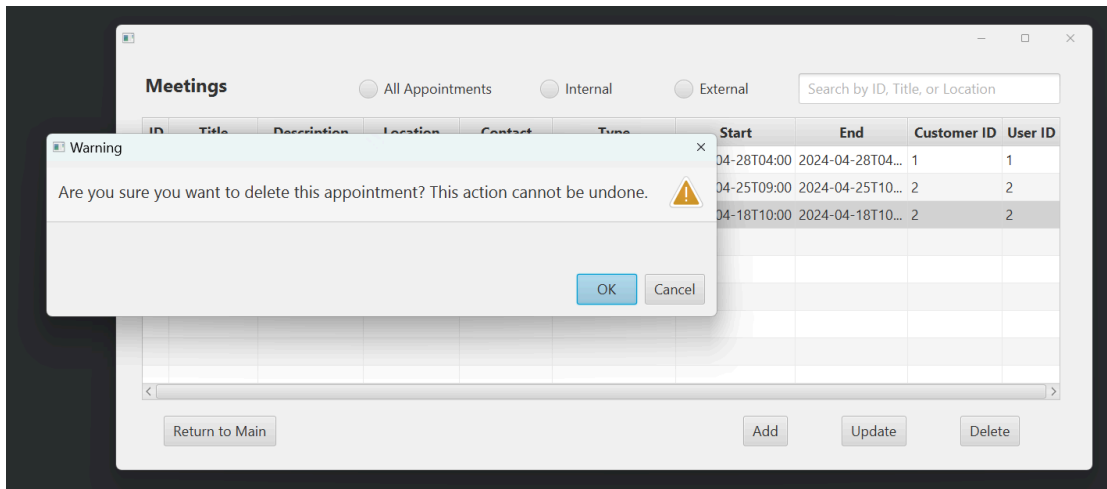


Deleting a Meeting

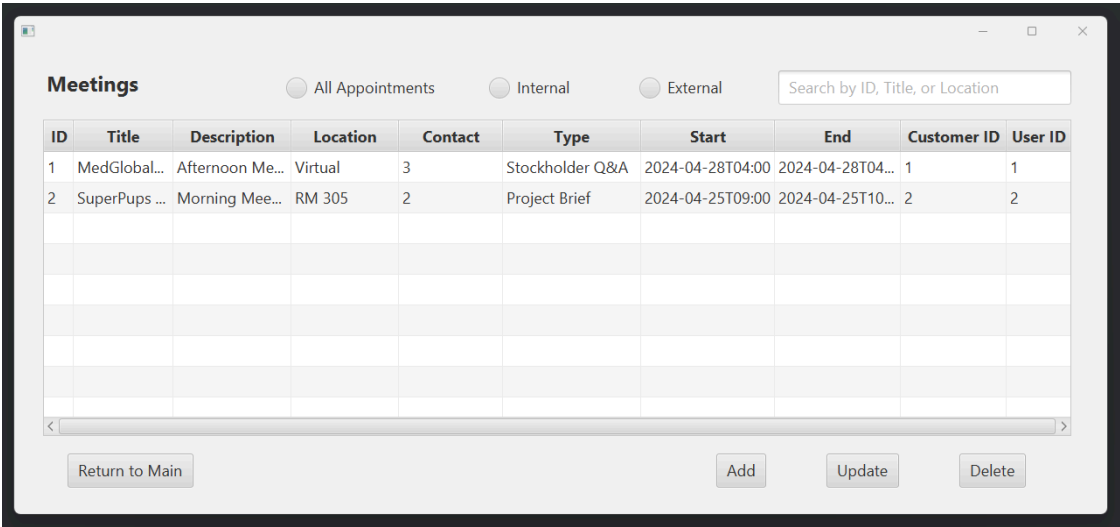
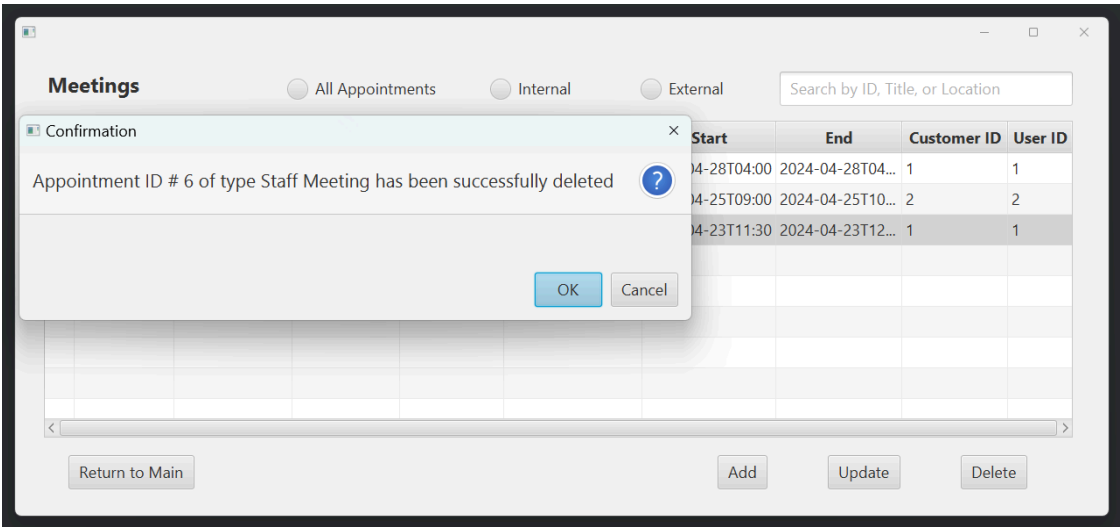
4. Select the Meeting you would like to delete



5. A confirmation prompt will appear. Select “OK” to permanently delete the Meeting.

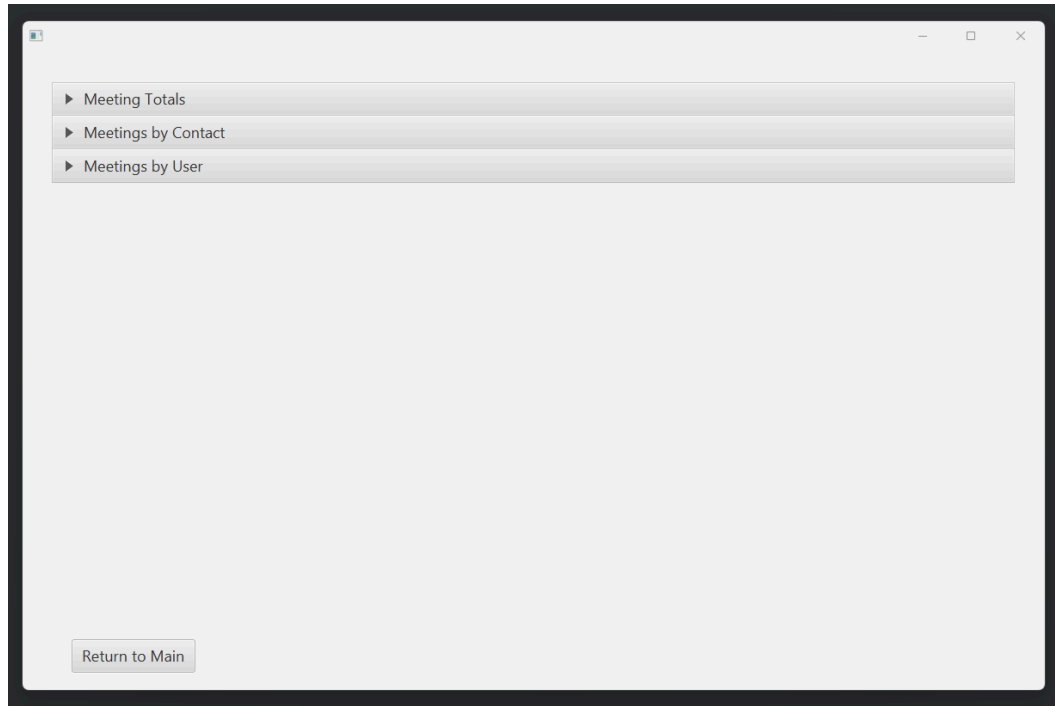


6. The Meeting will no longer appear in the Meeting table.

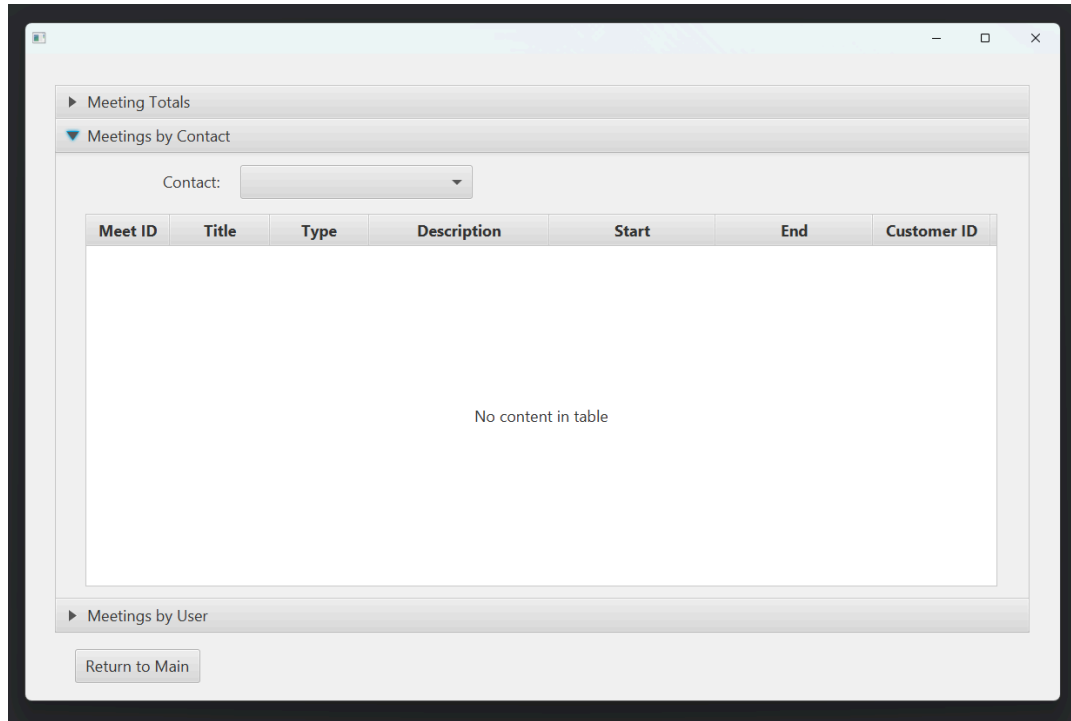


Reports

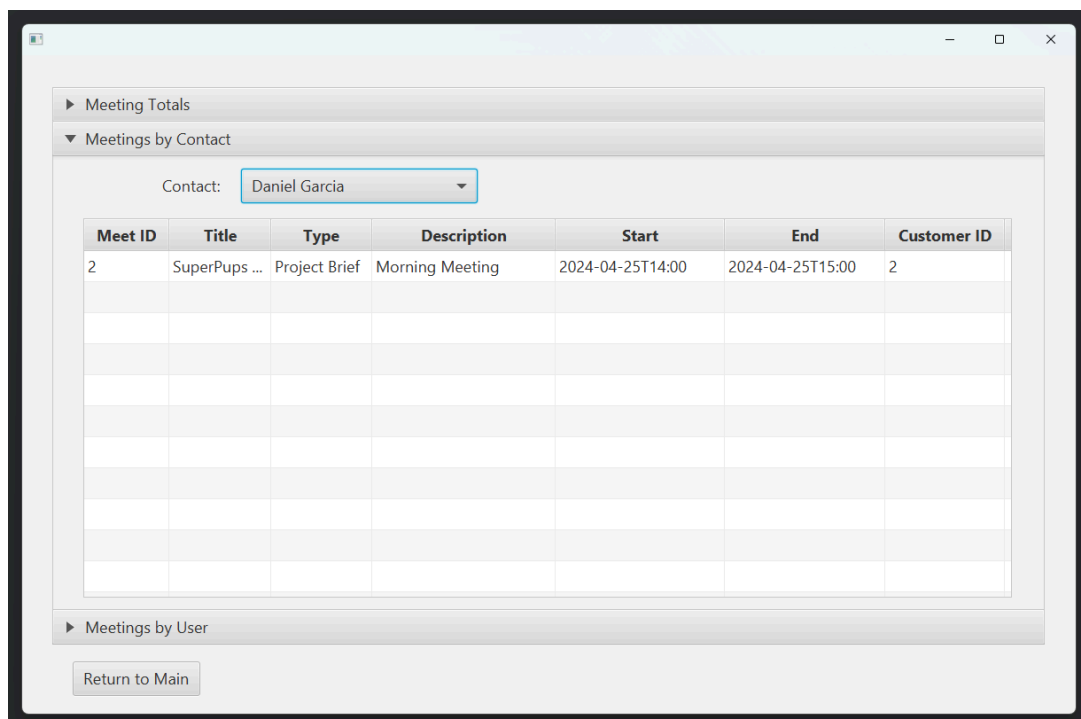
- 1. Clicking the “Reports” button in the Main Directory takes you to the Reports page.



2. *Selecting the “Meeting Totals” dropdown reveals two tables, Meeting Type and Meeting Month.*
 - a. *Meeting Type shows the number of scheduled meetings by type*
 - b. *Meeting Month shows the amount of scheduled meetings by month*
3. *Selecting the “Meetings by Contact” dropdown reveals an empty meeting table*



4. *Selecting a point of contact will show all meetings associated with that contact.*

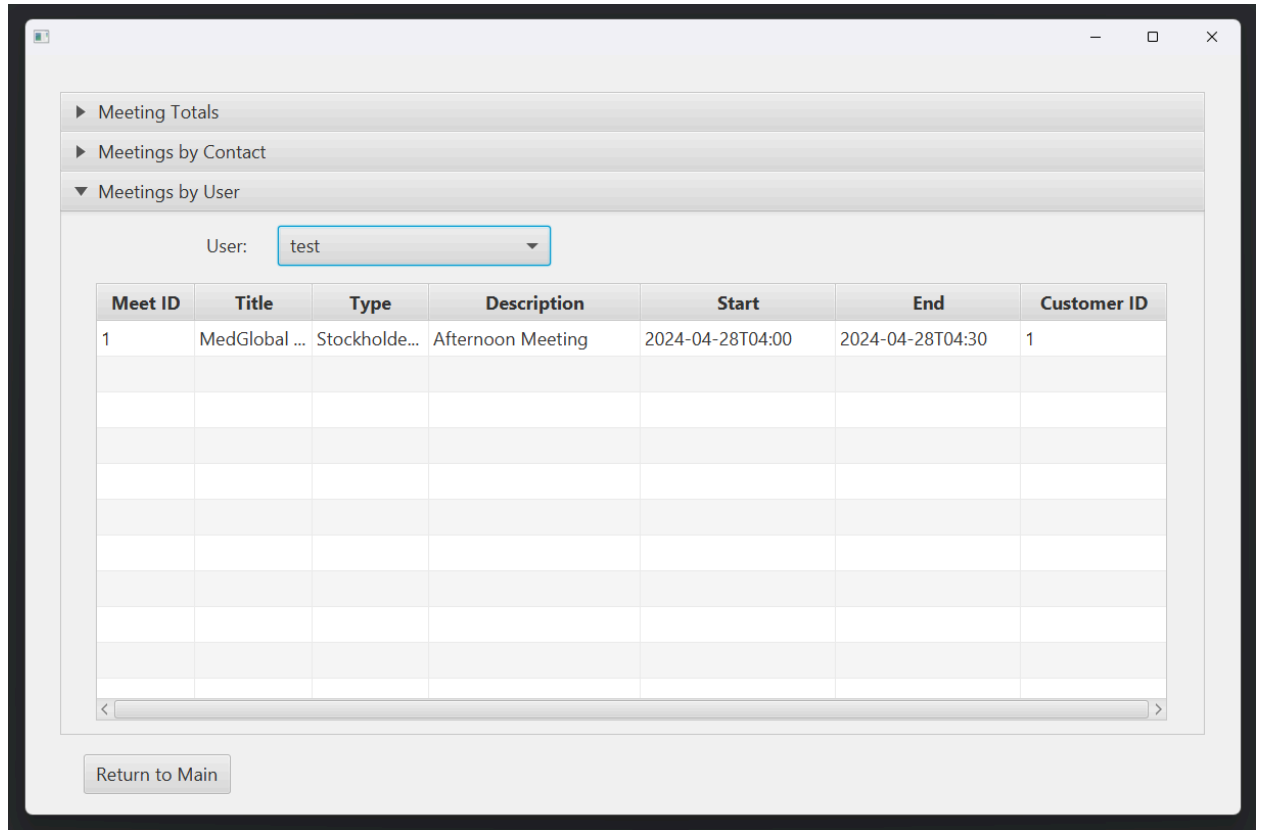


5. *Selecting the “Meetings by User” dropdown reveals an empty meeting table.*

The screenshot shows a web application window with a light gray background and a dark gray border. At the top, there are three expandable/collapsible sections: 'Meeting Totals' (expanded), 'Meetings by Contact' (expanded), and 'Meetings by User' (collapsed). Below these sections is a 'User:' label followed by a dropdown menu. Underneath the dropdown is a table with the following headers: 'Meet ID', 'Title', 'Type', 'Description', 'Start', 'End', and 'Customer ID'. The table body is empty, displaying the text 'No content in table'. At the bottom left of the window is a button labeled 'Return to Main'.

Meet ID	Title	Type	Description	Start	End	Customer ID
No content in table						

6. *Selecting a user will show all meetings associated with that user.*



7. Clicking the “Return to Main” button directs you back to the Main Directory screen.